

Experiment - 1

Aim: Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Hardware Required:

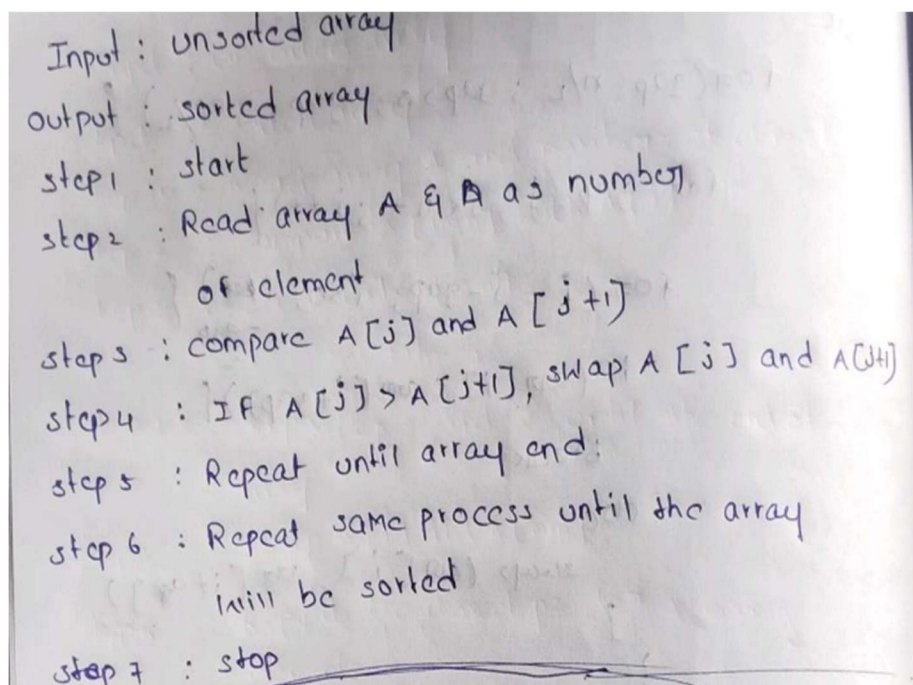
Mouse, keyboard.

Software Required:

code blocks.

a)Bubble sort

Algorithm:



Input : unsorted array
output : sorted array
step 1 : start
step 2 : Read array A & A as number
 of element
step 3 : compare $A[j]$ and $A[j+1]$
step 4 : IF $A[j] > A[j+1]$, swap $A[j]$ and $A[j+1]$
step 5 : Repeat until array end:
step 6 : Repeat same process until the array
 will be sorted
step 7 : stop

Program:

```
#include<iostream>
using namespace std;

class bubblesort
{
public:

int arr[100],size;

void arrayinput()
{
    cout<<"enter size of array";
    cin>>size;

    cout<<"enter array elements";

    int i;
    for(i=0;i<size;i++)
    {
        cin>>arr[i];
    }
}

void printarray()
{
    int i;
    for(i=0;i<size;i++)
    {
        cout<<arr[i]<<" ";
    }
}

void bubblelogic()
{
    int i,j,temp;

    for(i=0;i<size-1;i++)
    {
        for(j=0;j<size-i-1;j++)
        {
            if(arr[j]>arr[j+1])
            {
```

```
        temp=arr[j];
        arr[j]=arr[j+1];
        arr[j+1]=temp;
    }
}
};

int main()
{
    bubblesort bs;

    bs.arrayinput();

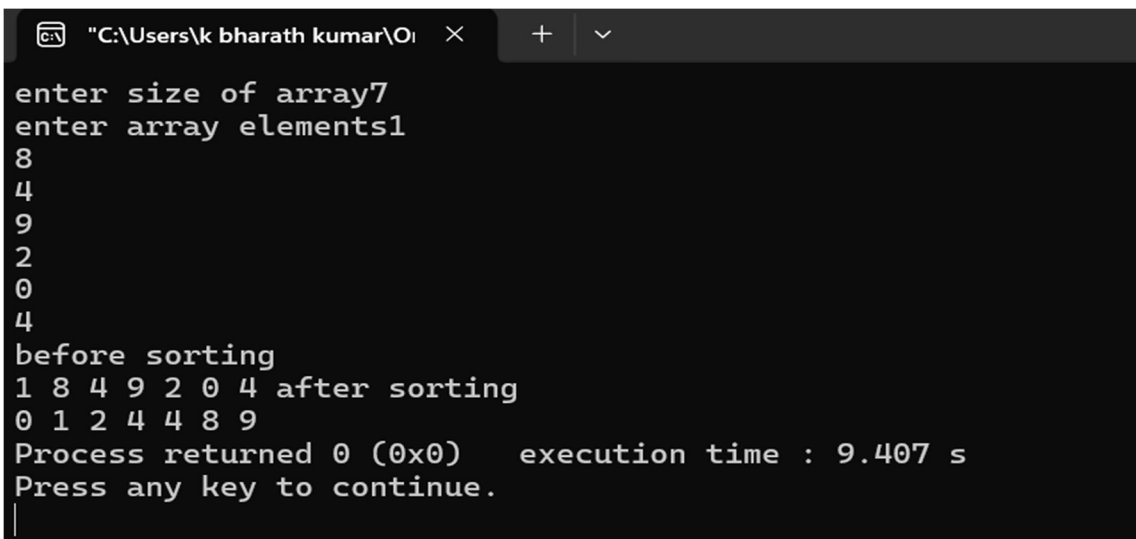
    cout<<"before sorting"<<endl;
    bs.printarray();

    bs.bubblelogic();

    cout<<"after sorting"<<endl;
    bs.printarray();

    return 0;
}
```

Output:



```
"C:\Users\k bharath kumar\O..." × + ▾
enter size of array7
enter array elements1
8
4
9
2
0
4
before sorting
1 8 4 9 2 0 4 after sorting
0 1 2 4 4 8 9
Process returned 0 (0x0)    execution time : 9.407 s
Press any key to continue.
|
```

Time complexity:

```

for(int i=0; i<n-1; i++) { — n-1
    for(j=0; j<n-i-1; j++) { — n-1
        if(A[j] > A[j+1]) { — 1
            temp = A[j]; — 1
            A[j] = A[j+1]; — 1
            A[j+1] = temp; — 1
        }
    }
}

```

$\therefore T(n) = (n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $= \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

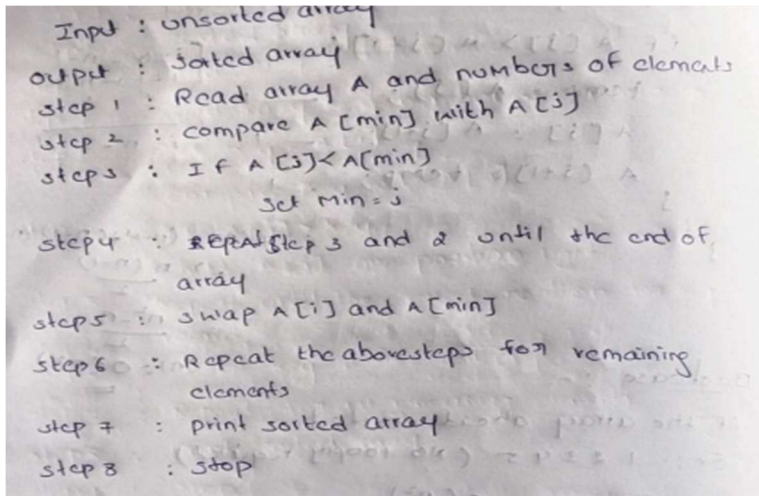
Best case :-
 if the array already sorted
 ex:- 1 2 3 4 5 (no looping required)
 $\Rightarrow T(n) = O(n)$

Average case :-
 for random arrangement of elements
 $= (n-1) + (n-2) + \dots + 1$
 $T(n) = O(n^2)$

Worst case
 IF the array is in reverse order
 ex:- 5 4 3 2 1
 $T(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} = O(n^2)$

b) Selection sort

Algorithm:



Input : unsorted array
Output : Sorted array
step 1 : Read array A and numbers of elements
step 2 : compare A [min] with A [j]
step 3 : IF A [j] < A [min]
Set min = j
step 4 : Repeat step 3 and 2 until the end of array
step 5 : swap A [i] and A [min]
step 6 : Repeat the above steps for remaining elements
step 7 : print sorted array
step 8 : stop

Program:

```
#include<iostream>
using namespace std;

class selectionsort
{
public:
    int arr[100],size;

    void arrayinput()
    {
        cout<<"enter size of array";
        cin>>size;

        cout<<"enter array elements";

        int i;
        for(i=0;i<size;i++)
        {
            cin>>arr[i];
        }
    }
    void printarray()
```

```
{
    int i;
    for(i=0;i<size;i++)
    {
        cout<<arr[i]<<" ";
    }
}

void selectionlogic()
{
    int i,j,min,temp;

    for(i=0;i<size-1;i++)
    {
        min=i;

        for(j=i+1;j<size;j++)
        {
            if(arr[j]<arr[min])
            {
                min=j;
            }
        }

        if(min!=i)
        {
            temp=arr[i];
            arr[i]=arr[min];
            arr[min]=temp;
        }
    }
}

};

int main()
{
    selectionsort ss;

    ss.arrayinput();

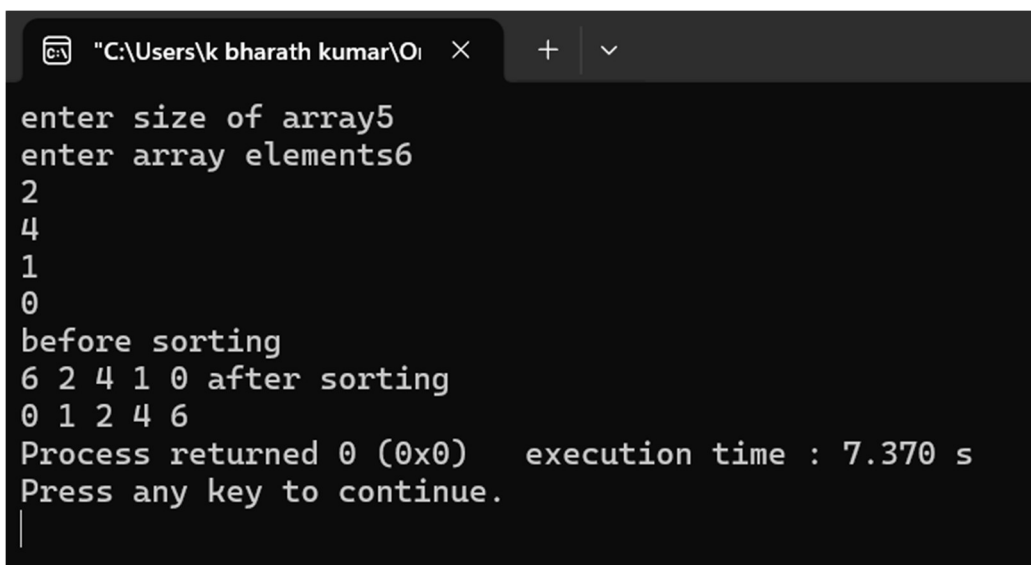
    cout<<"before sorting"<<endl;
    ss.printarray();

    ss.selectionlogic();
}
```

```
cout<<"after sorting"<<endl;
ss.printarray();

return 0;
}
```

Output:

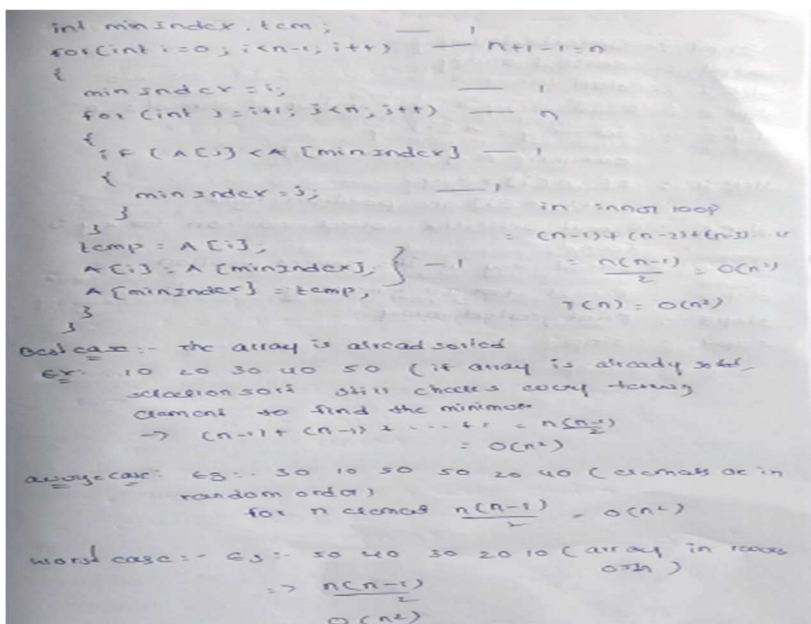


```

C:\Users\k bharath kumar\O... X + v
enter size of array5
enter array elements6
2
4
1
0
before sorting
6 2 4 1 0 after sorting
0 1 2 4 6
Process returned 0 (0x0)   execution time : 7.370 s
Press any key to continue.

```

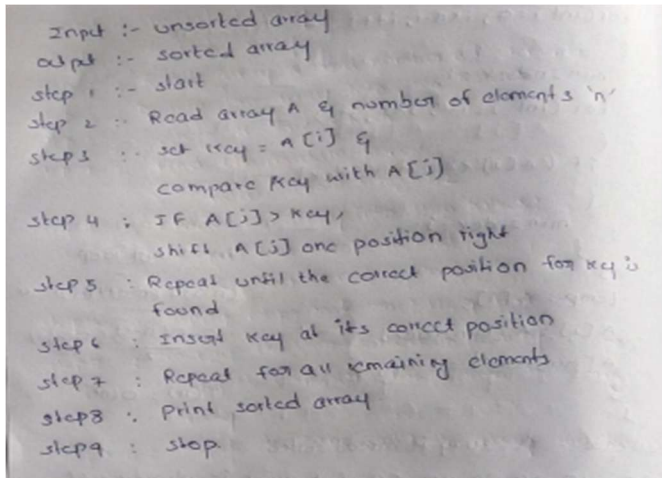
Time complexity:



$\text{int minIndex, temp;}$
 $\text{for (int i = 0; i < n-1; i++)}$ — $n-1$ times
 {
 $\text{minIndex} = i;$
 $\text{for (int j = i+1; j < n; j++)}$ — n times
 {
 $\text{if (A[j] < A[minIndex])}$ — 1
 {
 $\text{minIndex} = j;$
 }
 $\text{temp} = \text{A[i]};$
 $\text{A[i]} = \text{A[minIndex]};$
 $\text{A[minIndex]} = \text{temp};$
 }
 }
 Best case :- the array is already sorted
 eg:- 10 20 30 40 50 (if array is already sorted,
 selection sort still checks every element
 to find the minimum
 $\rightarrow (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$
 $= O(n^2)$
 Average case :- eg:- 30 10 50 50 20 40 (elements are in
 random order)
 for n elements $\frac{n(n-1)}{2} = O(n^2)$
 Worst case :- eg:- 50 40 30 20 10 (array in reverse
 order)
 $\Rightarrow \frac{n(n-1)}{2}$
 $= O(n^2)$

c) Insertion sort

Algorithm:



Input :- unsorted array
Output :- sorted array
step 1 :- start
step 2 :- Read array A & number of elements 'n'
step 3 :- set key = A[i] &
compare key with A[i]
step 4 :- IF A[i] > key,
shift A[i] one position right
step 5 :- Repeat until the correct position for key is found
step 6 :- Insert key at its correct position
step 7 :- Repeat for all remaining elements
step 8 :- Print sorted array
step 9 :- stop

Program:

```
#include<iostream>
using namespace std;

class insertionsort
{
public:
    int arr[100],size;

    void arrayinput()
    {
        cout<<"enter size of array";
        cin>>size;

        cout<<"enter array elements";

        int i;
        for(i=0;i<size;i++)
        {
            cin>>arr[i];
        }
    }
}
```



```
void printarray()
{
    int i;
    for(i=0;i<size;i++)
    {
        cout<<arr[i]<<" ";
    }
}

void insertionlogic()
{
    int i,j,temp;

    for(i=1;i<size;i++)
    {
        temp=arr[i];
        j=i-1;

        while(j>=0 && arr[j]>temp)
        {
            arr[j+1]=arr[j];
            j--;
        }

        arr[j+1]=temp;
    }
}

int main()
{
    insertionsort is;

    is.arrayinput();

    cout<<"before sorting"<<endl;
    is.printarray();

    is.insertionlogic();

    cout<<"after sorting"<<endl;
    is.printarray();

    return 0;
}
```

Output:

```

"C:\Users\k bharath kumar\O... × + v
enter size of array7
enter array elements1
0
9
8
7
6
5
before sorting
1 0 9 8 7 6 5 after sorting
0 1 5 6 7 8 9
Process returned 0 (0x0)   execution time : 7.700 s
Press any key to continue.

```

Time complexity

for (i=0; i<(n-1); i++) — n-1

```

{
    int min = i; — 1
    for (j=i+1; j<n; j++) — n-1
    {
        if arr[j] < arr[min] — 1
        {
            min = j; — 1
        }
    }
    if (min != i) — 1
    {
        int temp = arr[i]; — 1
        arr[i] = arr[min]; — 1
        arr[min] = temp; — 1
    }
}

```

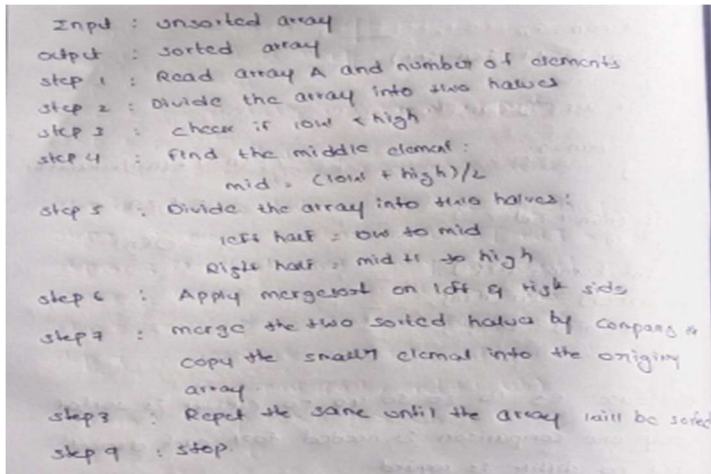
Best case: Eg: 10 20 30 40 50 (array is sorted)
only one comparison is needed for each element,
and no shifting is required
= $O(n)$

Average case: Eg: 20 50 20 40 30
Elements need to be compared and shifted to
their correct positions
= $O(n^2)$

Worst case: Eg: 50 40 30 20 10
Elements are in reverse order → element has to be
compared with all previous elements
= worst case = $O(n^2)$

d) Merge sort

Algorithm:



Input : unsorted array
Output : sorted array
step 1 : Read array A and number of elements
step 2 : Divide the array into two halves
step 3 : check if low < high
step 4 : Find the middle element :
 $mid = (low + high) / 2$
step 5 : Divide the array into two halves :
left half = low to mid
right half = mid + 1 to high
step 6 : Apply mergesort on left & right side
step 7 : merge the two sorted halves by comparing & copy the smallest element into the original array
step 8 : Repeat the same until the array will be sorted
step 9 : stop

Program:

```
#include<iostream>
using namespace std;

class mergesort
{
public:

int arr[100],size;

void arrayinput()
{
    cout<<"enter size of array";
    cin>>size;

    cout<<"enter array elements";

    int i;
    for(i=0;i<size;i++)
    {
        cin>>arr[i];
    }
}
```

```
void printarray()
{
    int i;
    for(i=0;i<size;i++)
    {
        cout<<arr[i]<<" ";
    }
}

void mergelogic(int low,int high)
{
    if(low<high)
    {
        int mid=(low+high)/2;

        mergelogic(low,mid);
        mergelogic(mid+1,high);

        int i=low,j=mid+1,k=0;
        int temp[100];

        while(i<=mid && j<=high)
        {
            if(arr[i]<arr[j])
            {
                temp[k]=arr[i];
                i++;
            }
            else
            {
                temp[k]=arr[j];
                j++;
            }
            k++;
        }

        while(i<=mid)
        {
            temp[k]=arr[i];
            i++;
            k++;
        }
        while(j<=high)
```

```
{
    temp[k]=arr[j];
    j++;
    k++;
}

for(i=low,k=0;i<=high;i++,k++)
{
    arr[i]=temp[k];
}
}
};

int main()
{
    mergesort ms;

    ms.arrayinput();

    cout<<"before sorting"<<endl;
    ms.printarray();

    ms.mergellogic(0,ms.size-1);

    cout<<"after sorting"<<endl;
    ms.printarray();

    return 0;
}
```

Output:

```

"C:\Users\k bharath kumar\O" x + v
enter size of array9
enter array elements
1
6
7
8
1
1
0
0
5
before sorting
1 6 7 8 1 1 0 0 5 after sorting
0 0 1 1 1 5 6 7 8
Process returned 0 (0x0)   execution time : 10.778 s
Press any key to continue.

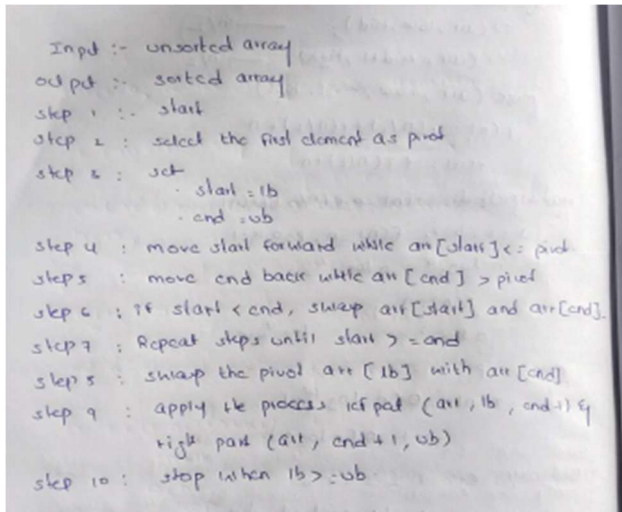
```

Time complexity:

merge sort
 $\Rightarrow (arr, low, mid); \quad \rightarrow T(n/2)$
 $\Rightarrow (arr, mid+1, high); \quad \rightarrow T(n/2)$
 $merge(arr, low, mid, high)$
 $T(n) = T(n/2) + T(n/2) + n$
 $T(n) = 2T(n/2) + n$
 master method: $T(n) = aT(n/b) + f(n)$
 $a=2, b=2, f(n)=n, p=0, k=1$
 $n \log_b a = n \log_2 2 = n$
 $f(n) = n$
 $n \log_2 2 = n \quad \log_2 2 = 1$
 Case 1: $p > -1$
 $T(n) = \Theta(n \log^p n)$
 $T(n) = \Theta(n \log n)$
 Best case Eg: 1 2 3 4 5 6
 The array already sorted still divides the array
 $= \Theta(n \log n)$
 Average case Eg: -4 2 6 1 5 3
 The array is divided into smallest parts and merged
 Average case $= \Theta(n \log n)$
 Worst case Eg: 6 5 4 3 2 1
 Array in reverse order still divides
 $= \Theta(n \log n)$

e)Quick sort

Algorithm:



Input :- unsorted array
output :- sorted array
step 1 :- start
step 2 :- select the first element as pivot
step 3 :- set
 - start = lb
 - end = ub
step 4 :- move start forward while arr[start] < pivot
step 5 :- move end back while arr[end] > pivot
step 6 :- if start < end, swap arr[start] and arr[end]
step 7 :- Repeat steps until start >= end
step 8 :- swap the pivot arr[lb] with arr[end]
step 9 :- apply the process recursively on left part (arr, lb, end-1) &
 right part (arr, end+1, ub)
step 10 :- stop when lb >= ub

Program:

```
#include<iostream>
using namespace std;

class quicksort
{
public:

    int arr[100], size;

    void arrayinput()
    {
        cout << "Enter size of array: ";
        cin >> size;

        cout << "Enter array elements: ";

        for(int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
}
```

```
void printarray()
{
    for(int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int partition(int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quicklogic(int low, int high)
{
    if(low < high)
    {
        int pi = partition(low, high);

        quicklogic(low, pi - 1);
        quicklogic(pi + 1, high);
    }
}

};
```



```
int main()
{
    quicksort qs;

    qs.arrayinput();

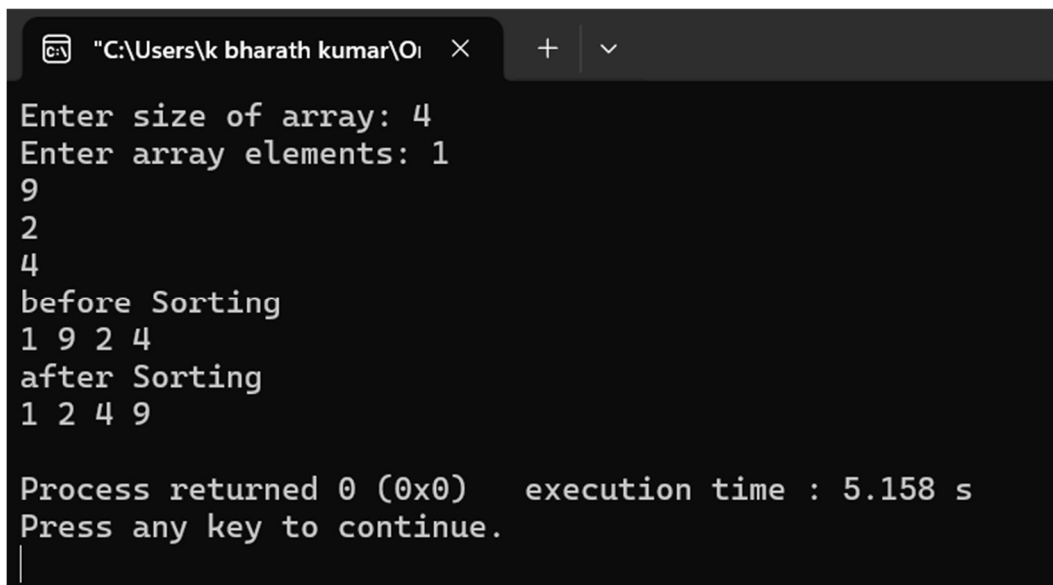
    cout << "before Sorting" << endl;
    qs.printarray();

    qs.quicklogic(0, qs.size - 1);

    cout << "after Sorting" << endl;
    qs.printarray();

    return 0;
}
```

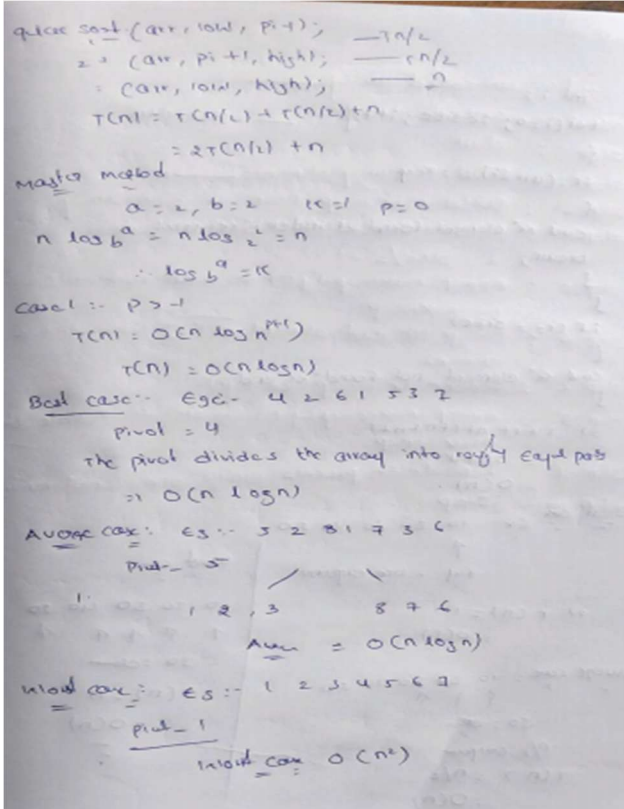
Output:



```
"C:\Users\k bharath kumar\O...  ×  +  v
Enter size of array: 4
Enter array elements: 1
9
2
4
before Sorting
1 9 2 4
after Sorting
1 2 4 9

Process returned 0 (0x0)   execution time : 5.158 s
Press any key to continue.
|
```

Time complexity:



Conclusion:

As per the experiment to Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort, Output was verified successfully according to the Experiment.